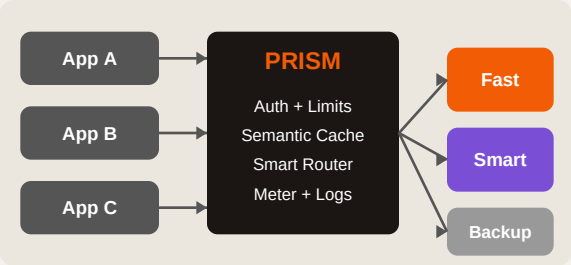

Prism: *Intelligent* LLM Gateway & Semantic Cache

CAPSTONE TOPIC: PRISM | APPROACH ONE-PAGER

Route smarter. Cache by meaning. Fail over gracefully.

WHO WE ARE

Prism is a production-grade LLM gateway: one OpenAI-compatible API in front of multiple model providers, handling keys, budgets, failover, caching and cost accounting for every tenant.

THE PROBLEM

Teams wire up providers one by one. Outages take products down, strong models answer trivial prompts, repeated questions are paid for twice, and nobody can see or cap spend per team.

THE SOLUTION

A single gateway where machine judgment sits in the request path: a difficulty scorer routes **auto** traffic to the right tier, and a semantic cache answers paraphrased questions without a provider call.

THE BENEFITS

- **Correct under load:** atomic reserve-and-settle budgets and rate limits give zero over-admission.
- **Resilient:** timeouts, backoff retries and an ordered fallback chain keep apps up when a provider fails.
- **Cost-smart:** cheap model for easy prompts, strong model for hard ones, cache for repeats.
- **Tenant-safe:** cache scoped per key, provider credentials never exposed to callers.
- **Observable:** every request logged (key, model, provider, tokens, cost, cache, fallback, latency).

OUR APPROACH: STEP BY STEP

1. **Load data and schema**
   Load model_pricing, seed_keys and gateway_config into SQLite (WAL mode). Keep seed key values and aliases (fast, smart, auto) untouched.
2. **Provider adapter and mocks**
   One adapter interface (chat, stream) with two mock providers that can inject latency, 429s, 500s or go fully down. Every upstream call has a timeout.
3. **Unified API and auth**
   POST /v1/chat/completions with virtual-key auth and alias resolution. Every response carries x-prism-provider, -cache, -fallback and -cost-usd.
4. **Streaming pass-through**
   Async SSE generator forwards chunks as they arrive, ends with data: [DONE], and records usage and cost in a finally block, even on client disconnect.
5. **Keys, limits and budgets**
   Model allowlist, RPM limit and monthly budget enforced in atomic DB transactions (reserve cost, settle after). Distinct errors: 401, 403, 429, 402.
6. **Routing, retries, failover**
   Alias maps to a primary model plus ordered fallback chain. Exponential backoff with jitter on transient errors, then fail over; flag and log it.
7. **Smart auto routing**
   Difficulty scorer on reasoning, code/math and constraint signals, not length. One-command eval over labeled cases proves it beats a length-only baseline.
8. **Semantic cache**
   Embed the prompt, match by cosine-similarity threshold, scope entries by (virtual key, model) so tenants never share. Track hit rate via header and logs.
9. **Metering and ops console**
   Cost = provider-reported tokens x price table. Log every request, expose a usage summary API and a simple console: usage by key, recent requests, hit rate.
10. **Verify and ship**
    Unit tests (cost, rate limit, cache, allowlist, routing), smoke_test, load_test (target: 0 over-admission), routing eval, README and demo video.

WHAT'S NEXT

Swap mocks for real providers via the same adapter, move rate limiting and cache to Redis and pgvector for multi-instance scale, add per-provider circuit breakers, and train a learned difficulty router from production logs.

See it work: run smoke_test.py, load_test.py and the routing eval, then watch the demo.

Seeded demo keys + mock providers: streaming, smart routing, budget rejection, rate limiting, cache hit, live failover.

github.com/Rohit-Sarkar55/Prism